PAOSITRA SMART ASSET

Guide d'explication du code

Pour répondre à : « montrez-moi le code », « comment avez-vous fait ça ? », « dans quel fichier ? »

Chaque section = un fichier réel du projet. Tu n'as pas besoin de retenir la syntaxe exacte,
juste de savoir OÙ chaque règle vit et POURQUOI, avec tes propres mots.

Comment utiliser ce document

Pour chaque fichier, tu trouveras :

Rubrique	À quoi elle sert
📍 Emplacement	Le chemin exact du fichier, pour l'ouvrir devant le jury sans chercher
Rôle	Une phrase pour dire à quoi sert le fichier, sans jargon
🗣️ Comment l'expliquer	Une formulation orale toute prête, en français simple
POURQUOI CE CHOIX	La justification technique, si on te demande "pourquoi comme ça et pas autrement ?"
QUESTION DU JURY	Une question probable + la réponse

🔤 VOCABULAIRE

Quelques mots que tu verras partout dans le code, expliqués une fois pour toutes :

- **classe** = un modèle qui décrit un type de donnée (ex. la classe `Article` décrit ce qu'est un article)
- **fonction** = un bloc de code qui fait une action précise, qu'on peut réutiliser
- **ForeignKey** = un lien vers UNE SEULE autre donnée (ex. un Article a UN SEUL TypeArticle)
- **ManyToManyField** = un lien vers PLUSIEURS autres données possibles des deux côtés
- **import** = "j'utilise un outil écrit ailleurs, dans un autre fichier"
- **transaction** = un bloc d'opérations qui doivent TOUTES réussir ensemble, ou TOUTES échouer ensemble

1. models.py — la description des données

`backend/inventaire/models.py`

Rôle : décrit chaque type de donnée du projet (Article, Entrée, Sortie...) et comment elles sont reliées entre elles. C'est la traduction directe du MCD en code.

Extrait — le modèle Utilisateur

```
class Utilisateur(AbstractUser):
    role = models.CharField(max_length=30, choices=ROLE_CHOICES)
    actif = models.BooleanField(default=True)

    def save(self, *args, **kwargs):
        self.is_active = self.actif
        super().save(*args, **kwargs)
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Ici, je ne repars pas de zéro : `AbstractUser` est un modèle déjà fourni par Django, qui gère déjà le mot de passe de façon sécurisée. J'ajoute juste deux informations propres à notre métier : le rôle, et si le compte est actif. La partie `def save(...)`, c'est une petite règle que j'ai ajoutée : à chaque fois qu'on enregistre un utilisateur, je force `is_active` à prendre la même valeur que `actif` — sinon Django ignorerait complètement notre champ actif pour décider qui peut se connecter."

POURQUOI CE CHOIX

Django utilise son propre champ `is_active`, pas le nôtre, pour bloquer une connexion. Sans cette ligne, désactiver un compte n'aurait eu aucun effet réel.

Extrait — le modèle Mouvement et sa règle de cohérence

```
article = models.ForeignKey(Article, ..., null=True, blank=True)
type_article = models.ForeignKey(TypeArticle, ..., null=True, blank=True)
quantite = models.PositiveIntegerField(null=True, blank=True)

def clean(self):
    if self.article and self.type_article:
        raise ValidationError("...jamais les deux...")
    if not self.article and not self.type_article:
        raise ValidationError("...jamais aucun des deux...")
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Un mouvement peut concerner soit un article précis, soit un type de produit avec une quantité — jamais les deux en même temps, jamais aucun des deux. La fonction `clean`, c'est le gardien de cette règle : avant d'enregistrer un mouvement, Django vérifie ces quatre cas, et refuse si un mouvement essaie de tricher."

QUESTION DU JURY — "Pourquoi ne pas avoir deux tables Mouvement séparées, une pour les articles, une pour les types ?"

RÉPONSE : Une seule table permet de retrouver tout l'historique d'un coup, sans avoir à interroger deux endroits différents et les recombiner. Le compromis, c'est cette vérification manuelle dans `clean()` plutôt qu'une contrainte automatique de la base de données.

Où trouver chaque entité du MCD

Entité MCD	Ligne approximative dans models.py
UTILISATEUR	Tout en haut du fichier
CATEGORIE, FOURNISSEUR, DIRECTION	Juste après, ce sont les plus courtes
TYPE_ARTICLE	Après Direction
ENTREE, LIGNE_ENTREE	Au milieu
ARTICLE	Juste après Ligne_entree
BON_SORTIE, INTERVIENT, LIGNE_BSORTIE	Ensuite
SORTIE, LIGNE_SORTIE	Puis

MOUVEMENT	Tout en bas, la dernière classe

2. services.py — la logique métier (LE fichier le plus important)

`backend/inventaire/services.py`

Rôle : c'est ici, et nulle part ailleurs, que sont écrites les règles métier réelles — que faire quand un produit entre en stock, que faire quand il sort. Si le jury demande "où est codée la règle X", la réponse est presque toujours : ici.

Fonction 1 : enregistrer_ligne_entree

```
def enregistrer_ligne_entree(entree, type_article, quantite, utilisateur):
    with transaction.atomic():
        ligne = LigneEntree.objects.create(...)
        type_article.quantite_stock += quantite
        type_article.save(update_fields=['quantite_stock'])

        if type_article.gestion_individuelle:
            for i in range(quantite):
                article = Article.objects.create(...)
                Mouvement.objects.create(type_mouvement='entree', article=article, ...)
        else:
            Mouvement.objects.create(type_mouvement='entree', type_article=type_article, quantite=quantite, ...)
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Cette fonction répond à la question : que se passe-t-il quand un produit arrive au CAME ? D'abord, dans tous les cas, le stock global du type augmente. Ensuite, deux chemins possibles : si le type est individualisé — `for i in range(quantite)`, ça veut dire 'répète autant de fois que la quantité' — on crée un Article physique et son mouvement à chaque tour de boucle. Sinon, un seul mouvement suffit, avec la quantité dessus."

POURQUOI CE CHOIX

`with transaction.atomic()` : tout le bloc réussit ensemble, ou rien n'est écrit du tout. Si la création du 3e article sur 5 échouait pour une raison quelconque, les 2 premiers ne resteraient pas non plus enregistrés — pas de données à moitié cohérentes.

Fonction 2 : effectuer_sortie

```
def effectuer_sortie(bon_sortie, date_sortie, ligne_bsortie_list, utilisateur):
    with transaction.atomic():
        for type_article, quantite in ligne_bsortie_list:
            if quantite > stock_disponible(type_article):
                raise ValidationError("Sortie refusée...")

        sortie = Sortie.objects.create(...)
        for type_article, quantite in ligne_bsortie_list:
            type_article.quantite_stock -= quantite
            ...
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"C'est la fonction la plus importante à savoir expliquer. Premier temps : je vérifie TOUTES les lignes demandées, sans rien écrire encore. Si une seule dépasse le stock disponible, `raise ValidationError` — ça veut dire 'j'arrête tout et je remonte une erreur' — et comme on est dans un bloc `transaction.atomic()`, absolument rien n'est enregistré, même pas la Sortie elle-même. Deuxième temps, seulement si tout est bon : je crée réellement la Sortie et je diminue le stock."

🐞 CE FICHIER CONTIENT UNE CORRECTION DE BUG

Cette fonction a été réécrite une fois en cours de projet : au début, la Sortie était créée AVANT la vérification, puis supprimée si ça dépassait — cette suppression a planté et laissé une trace corrompue en base. La version actuelle vérifie d'abord, écrit ensuite. C'est un bon exemple à raconter si le jury demande "avez-vous rencontré des difficultés ?".

QUESTION DU JURY — "Pourquoi cette logique n'est-elle pas directement dans la vue de l'API ?"

RÉPONSE : Pour qu'elle soit utilisée exactement de la même façon, peu importe qui l'appelle — l'API, l'admin, ou un futur import en masse. Un bug qu'on a trouvé (l'admin qui contournait cette fonction) a montré concrètement pourquoi c'est important.

3. admin.py — l'interface d'administration

`backend/inventaire/admin.py`

Rôle : configure ce que tu vois et peux faire dans l'interface `/admin/` de Django — l'écran que tu utilises pour tes démonstrations.

Extrait — pourquoi Entree est particulier

```
class EntreeAdmin(admin.ModelAdmin):
    inlines = [LigneEntreeInline]

    def save_formset(self, request, form, formset, change):
        if formset.model != LigneEntree:
            return super().save_formset(request, form, formset, change)
        instances = formset.save(commit=False)
        for obj in instances:
            if obj.pk is None:
                services.enregistrer_ligne_entree(...)
            else:
                obj.save()
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Par défaut, l'admin de Django sauvegarderait une ligne d'entrée directement, sans jamais appeler ma fonction `services.py` — donc sans créer les articles. La fonction `save_formset`, c'est moi qui intercepte ce moment précis pour rediriger vers la bonne logique. `if obj.pk is None` veut dire 'si c'est une ligne toute nouvelle, jamais encore enregistrée'."

🐞 CE FICHIER CONTIENT UNE CORRECTION DE BUG

C'est exactement le premier bug trouvé pendant les tests réels : sans cette interception, ajouter une ligne depuis l'admin créait la ligne mais aucun article ni mouvement.

4. serializers.py — traduire les données en JSON

`backend/inventaire/serializers.py`

Rôle : transforme les objets Python (Article, Entree...) en JSON pour l'API, et vice-versa quand on reçoit une requête.

```
class TypeArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = TypeArticle
        fields = ['id', 'nom_type_article', 'quantite_stock', ...]
        read_only_fields = ['quantite_stock']
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

" `fields` liste ce que l'API expose. `read_only_fields` est important : ça veut dire que `quantite_stock` peut être LU par l'API, mais jamais MODIFIÉ directement par quelqu'un qui enverrait une requête — il ne peut changer qu'en passant par `services.py`, jamais à la main."

POURQUOI CE CHOIX

Si n'importe qui pouvait écrire directement une valeur de stock via l'API, on perdrait toute garantie que le stock reflète vraiment les entrées et sorties réelles.

5. views.py — les routes de l'API et leurs actions

`backend/inventaire/views.py`

Rôle : reçoit les requêtes HTTP (GET, POST...) et décide quoi faire — appeler services.py, renvoyer une erreur, renvoyer des données.

Extrait — l'action qui déclenche une sortie

```
@action(detail=False, methods=['post'])
def effectuer(self, request):
    bon_sortie = BonSortie.objects.get(pk=bon_sortie_id)
    if hasattr(bon_sortie, 'sortie'):
        return Response({'erreur': '...déjà une sortie...'}, status=400)
    try:
        sortie, _ = services.effectuer_sortie(...)
    except DjangoValidationError as e:
        return Response({'erreur': str(e)}, status=400)
    return Response(SortieSerializer(sortie).data, status=201)
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Cette fonction ne contient presque aucune logique métier elle-même — elle récupère les informations de la requête, appelle `services.effectuer_sortie`, et traduit le résultat en réponse HTTP : 201 si ça marche, 400 avec le message d'erreur exact si le stock est insuffisant. C'est volontaire : toute la vraie logique reste dans `services.py`."

🔌 VOCABULAIRE

`@action` = une route spéciale en plus des routes automatiques (créer/lire/modifier/supprimer). `status=400` = "erreur causée par la demande elle-même" (ex. stock insuffisant). `status=201` = "créé avec succès".

6. urls.py (les deux fichiers) — le branchement des adresses

`backend/inventaire/urls.py  +  backend/config/urls.py`

Rôle : associe chaque adresse web (ex. /api/sorties/) à la vue qui doit répondre.

```
router.register('sorties', views.SortieViewSet)
# => génère automatiquement /api/sorties/, /api/sorties/{id}/, /api/sorties/effectuer/
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Je n'écris pas chaque adresse à la main. `router.register` génère automatiquement toutes les adresses standard pour un type de donnée, plus les actions spéciales comme `effectuer`. Dans `config/urls.py`, une seule ligne branche tout ça sous le préfixe `/api/`."

7. tests.py — la preuve automatisée que tout fonctionne

`backend/inventaire/tests.py`

Rôle : 12 vérifications automatiques qui recréent des données de test, exécutent une action, et vérifient que le résultat est exactement celui attendu.

```
def test_07_stock_insuffisant_bloque_sans_rien_ecrire(self):
    services.enregistrer_ligne_entree(self._entree('E7'), self.type_encre, 10, self.user)
    bs = self._bon_sortie('BS7', self.type_encre, 999)
    with self.assertRaises(ValidationError):
        services.effectuer_sortie(bs, date.today(), [(self.type_encre, 999)], self.user)
    self.type_encre.refresh_from_db()
    self.assertEqual(self.type_encre.quantite_stock, 10, "le stock ne doit pas avoir bougé")
```

🗣 COMMENT L'EXPLIQUER SIMPLEMENT

"Chaque test suit le même schéma : je prépare une situation (ici, 10 cartouches en stock), je fais une action (demander d'en sortir 999), et je vérifie deux choses avec `assertRaises` et `assertEqual` — que ça a bien été refusé, ET que le stock n'a pas bougé d'un chiffre malgré la tentative."

POURQUOI CE CHOIX

Un test automatisé s'exécute en quelques secondes et peut être relancé à volonté — bien plus fiable que de re-tester à la main à chaque modification du code.

QUESTION DU JURY — "Que se passe-t-il si vous modifiez le code plus tard ?"

RÉPONSE : Je relance `python manage.py test inventaire` . Si un test échoue, ça veut dire que la modification a cassé une règle qui marchait avant — je le sais immédiatement, sans devoir tout re-tester à la main dans l'admin.

Récapitulatif — "dans quel fichier je trouve...?"

Si on te demande où est codé...	Fichier
La structure d'une table (champs, relations)	models.py
La règle "stock insuffisant = bloqué"	services.py, fonction effectuer_sortie
La règle "individualisé vs consommable"	services.py, fonction enregistrer_ligne_entree
Ce qui s'affiche dans /admin/	admin.py
Ce que l'API renvoie en JSON	serializers.py
Ce qui se passe quand une requête arrive	views.py
La liste des adresses de l'API	inventaire/urls.py + config/urls.py
La preuve que ça fonctionne	tests.py